C++ STL Cheatsheet

Containers

Vector (vector)

Operations: Access $O(1)$, Insert/Delete at end $O(1)$ amortized, elsewhere $O(n)$.
Note: `vector` is implemented as a dynamic array.

```
#include <vector>
std::vector<int> vec; // Declaration
vec.push_back(10); // Insert at the end - Amortized O(1)
vec.size(); // Get the number of elements - O(1)
vec.insert(vec.begin() + 1, 20); // Insert at index 1 - O(n)
vec.erase(vec.begin() + 1); // Remove at index 1 - O(n)
vec.pop_back(); // Remove the last element - O(1)
vec.clear(); // Remove all elements - O(n)
```

List (list)

Operations: Insert/Delete anywhere $O(1)$ (with iterator), Access $O(n)$.
Note: `list` is a doubly-linked list.

```
#include <list>
std::list<int> l;
l.push_back(10); // Insert at end - O(1)
l.push_front(5); // Insert at front - O(1)
```

Set (set)

Operations: Insert/Delete/Search $O(\log n)$.
Note: `set` is a balanced binary search tree (Red-Black Tree).

```
#include <set>
std::set<int> s;
s.insert(1); // Insert an element - O(log n)
s.erase(1); // Remove an element - O(log n)
s.find(1); // Find an element - O(log n)
s.clear(); // Remove all elements - O(n)
s.size(); // Get the number of elements - O(1)
```

Map (map)

Operations: Insert/Delete/Search by key $O(\log n)$.
Note: `map` is implemented as a balanced binary search tree.

```
#include <map>
std::map<int, int> m;
m[1] = 2; // Insert or update a key-value pair - O(log n)
m.emplace(1, 2); // Insert a key-value pair - O(log n)
m.at(1); // Access element by key with bounds checking - O(log n)
m.find(1); // Find an element by key - O(log n)
m.erase(1); // Remove an element by key - O(log n)
m.clear(); // Remove all elements - O(n)
m.size(); // Get the number of elements - O(1)
```

Unordered Set (unordered_set)

Operations: Insert/Delete/Search (Average: $O(1)$, Worst: $O(n)$).
Note: `unordered_set` is implemented as a hash table.

```
#include <unordered_set>
std::unordered_set<int> us;
us.insert(1); // Insert an element
us.find(1); // Find an element
us.erase(1); // Remove an element
us.clear(); // Remove all elements
us.size(); // Get the number of elements
```

Unordered Map (unordered_map)

Operations: Insert/Delete/Search by key (Average: $O(1)$, Worst: $O(n)$).
Note: `unordered_map` is implemented as a hash table.

```
#include <unordered_map>
std::unordered_map<int, int> um;
um[1] = 2; // Insert or update a key-value pair
um.emplace(1, 2); // Insert a key-value pair
um.find(1); // Find an element by key
um.erase(1); // Remove an element by key
um.clear(); // Remove all elements
um.size(); // Get the number of elements
```

Stack (stack)

Operations: Push/Pop/Top $O(1)$.
Note: `stack` is implemented as a deque.

```
#include <stack>
std::stack<int> st;
st.push(10); // Push an element - O(1)
st.pop(); // Pop an element - O(1)
st.top(); // Get the top element - O(1)
st.empty(); // Check if empty - O(1)
st.clear(); // Remove all elements - O(n)
st.size(); // Get the number of elements - O(1)
```

Queue (queue)

Operations: Enqueue/Dequeue/Front $O(1)$.
Note: `queue` is implemented as a deque.

```
#include <queue>
std::queue<int> q;
q.push(10); // Enqueue an element - O(1)
q.pop(); // Dequeue an element - O(1)
q.front(); // Get the front element - O(1)
q.size(); // Get the number of elements - O(1)
q.empty(); // Check if empty - O(1)
q.clear(); // Remove all elements - O(n)
```

Priority Queue (priority_queue)

Operations: Insert/Delete Max $O(\log n)$, Get Max $O(1)$.
Note: `priority_queue` is implemented as a heap.

```
#include <queue>
std::priority_queue<int> pq;
pq.push(10); // Insert an element - O(log n)
pq.pop(); // Remove the top element - O(log n)
pq.top(); // Get the top element - O(1)
pq.size(); // Get the number of elements - O(1)
pq.empty(); // Check if empty - O(1)
pq.clear(); // Remove all elements - O(n)
```

Algorithms

Sort

Time Complexity: $O(n \log n)$.
Note: By default, it sorts in ascending order, algorithm used is IntroSort.

```
#include <algorithm>
std::sort(vec.begin(), vec.end()); // Ascending
std::sort(vec.begin(), vec.end(), std::greater<int>()); // Descending
```

Max Element and Min Element

Time Complexity: $O(n)$.
Note: Returns an iterator to the maximum/minimum element.

```
auto max_it = std::max_element(vec.begin(), vec.end());
auto min_it = std::min_element(vec.begin(), vec.end());
```

Utilities

Iterators

Traverse containers or find elements.

```
auto it = vec.begin(); // Iterator to first element
auto end = vec.end(); // Iterator to one past last
auto f_it = std::find(vec.begin(), vec.end(), 10);
```

Enum Class

Strongly typed enumerations.

```
enum class Color { Red, Green, Blue };
Color c = Color::Red; // Access using EnumClass::Option
```

Casting (static_cast, dynamic_cast)

```
// Compile-time type conversion
int a = static_cast<int>(3.14);
// Run-time conversion for polymorphic types (downcasting)
Base* b = new Derived();
Derived* d = dynamic_cast<Derived*>(b);
```

Smart Pointers (memory)

Replaces manual memory management.
Note: Introduced in C++11.

```
#include <memory>
// Unique pointer (exclusive ownership)
std::unique_ptr<int> p1 = std::make_unique<int>(10);

// Shared pointer (shared ownership, reference counted)
std::shared_ptr<int> p2 = std::make_shared<int>(20);
```

Pair (pair)

Simple container to store two values.

```
#include <utility>
std::pair<int, int> p = {1, 2};
int a = p.first;  // Access first element
int b = p.second; // Access second element
```

Tuple (tuple)

Generalization of pair to hold more than two items.

```
#include <tuple>
std::tuple<int, char, double> t = {1, 'a', 2.0};
```

Swap

Exchange the values of two variables. $O(1)$.

```
std::swap(a, b);
```

Reverse

Reverse the elements of a container. $O(n)$.

```
std::reverse(vec.begin(), vec.end());
```

String Stream

Used to manipulate strings as if they were input/output streams.

```
#include <sstream>
std::stringstream ss;
ss << "Hello";
ss >> str;
```

String Functions

```
std::stoi("10"); // convert to int
std::to_string(10);
std::getline(std::cin, str);
```

Count

Count the number of occurrences of a value in a range.

```
std::count(vec.begin(), vec.end(), val);
```